

Dexter

Architecture reviews, automated end to end

Product capabilities as at 5 August 2026

Dexter assesses a solution design against your organisation's own standards, and produces a compliance report in which every finding is traceable to the clause it came from and the passage it rests on.

It is not a chatbot over your documents. The question is never typed by a person: every clause in your standards library is fired at the submitted design in turn, which is what makes the result a complete review rather than an answer to whatever somebody thought to ask.

Three things it knows that a generic tool cannot

1. Your standards. The framework is built from the documents you upload, not from a library of best practice.
2. Your decisions. A ruling made once applies to every assessment that follows.
3. Your technology. Findings are written against the platforms you actually run.

The rest of this document sets out what the system does today, how a document moves through it, and the checks that keep its output honest.

How a document moves through Dexter

Six stages, fully automated. A submission is uploaded once; everything after that runs without a person in the loop until there is a report to review.

1

Upload

A PDF is dropped into one of two shelves: a reference standard, or a design to be assessed. The shelf it lands in is what it becomes - there is no mode to set wrongly.

2

Parse

The document is broken into its real structure: sections, clauses, tables and figures, with page numbers preserved. Diagrams are read by a vision model and described in text. The contents page is reconciled against what was found, and anything that does not line up is raised for review.

3

Chunk

Each clause becomes one indexed unit, never split. Tables are indexed row by row with their headers attached. The document's own structure sets the boundaries, so a requirement is never severed from the rule it belongs to.

4

Index

Every unit is embedded as a vector and stored alongside a full-text index. Retrieval uses both and fuses the rankings, because the terms that matter here - TLS 1.2, RPO, MFA - are exactly the ones a vector search alone handles poorly.

5

Assess

Each clause of your framework is searched against the submitted design and judged in turn. The verdict is one of five, and must cite the passage it rests on.

6

Review

An architect works the findings worst-first, confirming or overriding each one. An override can be kept as a standing decision, which is where the system starts to learn.

The report

Every finding carries: the clause it answers, one of five verdicts, the model's reasoning, and the exact passage in the submitted document that supports it - with the page number. A finding that cites nothing is not shown as a verdict. It is escalated to a person.

What a finding says

Every clause resolves to exactly one of five verdicts.

Verdict	Meaning
Covered	Every requirement in the clause is met by the design.
Partial	The clause is addressed, but a requirement is left unmet.
Contradicts	The design states something the standard forbids.
Absent	The design does not address this clause at all.
Needs review	The evidence was inconclusive. A person has to decide.

Institutional memory

When an architect overrides a finding, they say why. That reason can be kept as a standing decision, and every assessment from then on is judged with it in view. A question your organisation has answered once is not asked again.

- **Each decision** carries an effect: it either accepts an arrangement as satisfying a clause, rejects it, or supplies background.
- **Nothing is edited** in place. Revising one writes a new version and keeps the old, so a report from last year still means what it said.
- **A decision** can carry an end date, because an architecture exception granted for twelve months should expire on its own.
- **Two decisions that disagree** on one clause are never resolved by taking the most recent. They are surfaced, because a register that contradicts itself is a governance problem you need to see.

Your technology, in every finding

Your technology reference - cloud, warehouse, BI, source systems, the platforms you run and the ones you are evaluating - is read on every assessment. It is context, never a rule: it makes a recommendation concrete rather than generic, but a verdict must still rest on what the design actually says.

How we keep it honest

The failure that matters in compliance work is not a wrong answer a reviewer can catch. It is a confident wrong answer at scale. Dexter is built to refuse rather than to guess, and every refusal explains itself.

- **A verdict** that claims the design covers a clause must quote the passage that proves it. One that cites nothing is downgraded, because a claim about a document that cites nothing in it was invented.
- **A verdict** of 'absent' reached when the search found nothing relevant is downgraded. Nothing retrieved is not the same as nothing there.
- **A description** read from a diagram cannot decide a verdict alone. It is a model's reading of an image, not text from the page, and the diagram is shown beside the finding so a person can check it.
- **A verdict** citing a standing decision that does not exist is refused outright.
- **A section** that states an obligation but yields no clause is reported, not passed over - so a rule can never go unassessed in silence.

Traceability

Frameworks are versioned, so a finding that cites a clause still resolves after the standard is rewritten. Every run records the model that produced it. Every reviewer decision records who made it and when. Nothing in a past report changes silently.

Separation between clients

Each client organisation is a hard boundary. Every read of document, clause or vector data is checked against membership first; one client's material cannot surface in another's work.